5. Project Development Phase

5.1 Introduction

The development phase converts the design into a functional application. It involves implementing the frontend, backend services, database, authentication, and Google Gemini integration. The components are integrated through REST APIs to form a complete application.

5.2 Development Environment

Technology	Purpose
Python	Backend programming
FastAPI	REST API and backend framework
HTML / CSS	Web page structure and interface design
JavaScript	Client-side functionality and document editor
SQLite / PostgreSQL	Data storage
Google Gemini API	Drafting, clause explanation, and review
ReportLab / python-docx	PDF and Word export
JWT	User authentication
Visual Studio Code	Development environment
Git / GitHub	Source code management

5.3 Application Development Architecture

User Interface -> REST API -> FastAPI Backend -> Database, and Form Input -> Prompt Builder -> Gemini Model -> Response Parser -> Editor -> Export. This separates presentation, business logic, data management, and AI processing, making the application easier to maintain and extend.

5.4 Frontend Development

The frontend is built with HTML, CSS, and JavaScript. It includes login and registration pages, a document type selector, a step-by-step questionnaire, a clause-by-clause editor, a side panel for explanations and review results, an export preview, and a history page. The interface is responsive, and loading indicators show when the draft is being prepared. A disclaimer is displayed on the editor and export pages.

5.5 Backend Development

The backend is implemented in Python using FastAPI. It exposes REST endpoints for authentication, templates, documents, clause regeneration, explanation, review, and export. Pydantic models validate request data, and dependency functions verify JWT tokens on protected routes.

5.6 Gemini AI Integration

The backend communicates with the Google Gemini API through its Python SDK. For drafting, a prompt template combines the document type, jurisdiction, and all user-supplied details and instructs the model to use only the supplied facts, to insert clearly marked placeholders for missing information, and to return a fixed JSON structure of numbered clauses. Separate prompts handle clause explanation in plain language and document review for missing clauses, ambiguity, and

one-sided terms. The backend parses the responses, handles errors, timeouts, and safety-blocked outputs, and returns clean results to the frontend.

5.7 Database Development

The database stores users, documents, clauses, versions, reviews, and a generation log as described in the design phase. SQLAlchemy or direct SQL queries are used for data access, and each record is linked to the user through a user identifier so that users can access only their own data.

5.8 Authentication Implementation

Registration stores a hashed password. Login verifies the credentials and issues a JWT token, which the frontend sends with each request. Protected endpoints reject missing or invalid tokens.

5.9 Challenges and Solutions

Challenge	Solution
AI inventing names, dates, or amounts	Prompt rules to use only supplied facts and mark missing items as placeholders
Inconsistent document structure from AI	Strict prompt instructions, JSON parsing with validation and fallback handling
Legal language hard to understand	Plain-language clause explanation feature
Documents may not suit every jurisdiction	Jurisdiction selected in the form and included in the prompt; visible disclaimer
Formatting of exported documents	Clause numbering and layout handled by ReportLab and python-docx templates
API errors, delays, and blocked outputs	Try/except handling, retries, safety-block messages, and loading indicators
Keeping API key and personal data secure	Environment variables, backend-only API calls, per-user data filtering